Property Empire - פרויקט FastAPI לתלמידים

מטרת הקובץ: זהו קובץ התחלה מלא לפרויקט כיתתי. התלמידים מתקדמים בחלקים, ובכל נקודת הגשה מוסיפים יכולת אחת או כמה יכולות קטנות. כל חלק נשען על החלקים הקודמים.

מבנה הפרויקט: צד שרת ב-FastAPI ותיקיית מנוע משחק טהורה ב-Python. ממשק ה-React המשותף מתקשר עם השרת רק דרך HTTP.

כלל חשוב: חוקי המשחק לא נכנסים ל-routes. הם נמצאים בשכבת השירותים של `game_engine` .

תיאור קצר של המשחק

בונים משחק לוח פשוט בסגנון רכישת נכסים. יש 2 עד 4 שחקנים, לוח עם משבצות, קובייה, כסף, תורות, רכישת נכסים, תשלום שכירות, משבצות בונוס/קנס/הפתעה, פשיטת רגל ומנצח.

הרצה מקומית

שרת FastAPI

```
cd student-api
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

ממשק React משותף

```
cd shared-ui
npm install
npm run dev
```

בדיקות

```
cd student-api
```

```
pytest
```

הנחיית עבודה: אחרי כל חלק מריצים בדיקות. אם בדיקה נכשלת, מתקנים לפני שממשיכים. אין לדלג על חלקים.

מפת תיקיות

תיקייה / קובץ	תפקיד
`app/main.py`	יצירת אפליקציית FastAPI וחיבור routes ו-exception handlers.
`app/api/game_routes.py`	נקודות HTTP בלבד: קבלת קלט, קריאה ל-service, החזרת response.
`app/schemas/game_schemas.py`	מודלים של Pydantic שמגדירים את חוזה ה-JSON מול הממשק.
`game_engine/models`	מחלקות הדומיין: שחקן, משחק, משבצות.
`game_engine/services`	חוקי המשחק והפעולות המרכזיות.
`game_engine/repositories`	שמירת מצב המשחק בזיכרון.
`game_engine/factories`	בניית לוח המשחק.
`tests/test_api.py`	בדיקות קבלה שמגדירות מתי כל חלק עובד.

חוזה API סופי

Endpoint	פעולה	תוצאה צפויה
`GET /api/health`	בדיקת חיים	`{"status": "ok"}`
`POST /api/game`	יצירת משחק	מצב משחק מלא
`GET /api/game`	שליפת המשחק הנוכחי	מצב משחק מלא
`POST /api/game/roll`	זריקת קובייה ותזוזה	`{"state": ...}`
`POST /api/game/buy`	קניית נכס	`{"state": ...}`
`POST /api/game/end-turn`	סיום תור	`{"state": ...}`
`POST /api/game/reset`	איפוס משחק	משחק במצב `not_started`

כללי הגשה

- כל חלק מוגש כ-checkpoint נפרד.
- בכל הגשה יש לציין אילו בדיקות עוברות.
- אין לשנות את ממשק ה-JSON בלי הוראה מפורשת, כי ה-React UI תלוי בו.
- אין לכתוב חוקי משחק בתוך `app/api/game_routes.py` .
- מותר להוסיף פונקציות עזר ושירותים אם הם שומרים על חלוקת האחריות.

Checkpoint 0 - הכרת הפרויקט והרצה

מטרה

להריץ את הפרויקט, להבין את מבנה התיקיות, ולוודא שהסביבה מוכנה.

משימות

1. פתחו את התיקיות `student-api` ו- `shared-ui` .
2. הריצו את שרת FastAPI.
3. הריצו את ממשק React וחברו אותו לכתובת `http://localhost:8000` .
4. הריצו `pytest` וקראו את שמות הבדיקות.

הגשה: צילום מסך של השרת רץ, צילום מסך של הממשק, ורשימת 3 קבצים שנראים לכם מרכזיים בפרויקט.

Checkpoint 1 - Health Check ומבנה FastAPI בסיסי

מטרה

לבנות endpoint ראשון ולהכיר את FastAPI, router ו-response model.

קבצים מרכזיים

- `app/main.py`
- `app/api/game_routes.py`
- `app/schemas/game_schemas.py`

משימות

1. צרו router לנתיב `/api` .
2. הגדירו endpoint בשם `health` שמחזיר status תקין.
3. צרו Pydantic response model בשם `HealthResponse` .
4. ודאו שהנתיב `GET /api/health` מחזיר `{"status": "ok"}` .

בדיקות קבלה

- `test_health` עוברת.
- פתיחת `/docs` בדפדפן מציגה את ה-endpoint.

> **הגשה:** קוד ה-route, צילום מסך של Swagger, ותוצאת הבדיקה.

Checkpoint 2 - מודלים ראשונים ויצירת משחק

מטרה

ליצור את אובייקטי הדומיין הראשונים: שחקן, משחק ולוח בסיסי.

קבצים מרכזיים

- `game_engine/models/player.py`
- `game_engine/models/game.py`
- `game_engine/factories/board_factory.py`
- `game_engine/services/game_service.py`

משימות

1. צרו מחלקת `Player` עם id, name, money, position, bankrupt flag ורשימת נכסים.
2. צרו מחלקת `Game` עם players, board, current player, status ו-actions.
3. צרו factory שמייצר לוח של 16 משבצות לפחות.
4. ממשו `create_game` שמקבל שמות שחקנים ומחזיר מצב משחק.
5. הוסיפו `POST /api/game` endpoint.

חוזה קלט

```
{
  "player_names": ["Ada", "Linus"]
}
```

בדיקות קבלה

- סטטוס המשחק הוא `active`.
- השחקן הנוכחי הוא id 1.
- הפעולה הזמינה הראשונה היא `roll`.
- יש 2 עד 4 שחקנים ולוח בגודל 16.

הגשה: קבצי המודלים, factory, route, ותוצאת `test_create_game`.

Checkpoint 3 - ולידציה וחוזה JSON

מטרה

להגן על ה-API מפני קלט לא תקין ולייצב את צורת התשובה לממשק.

משימות

1. ב-`CreateGameRequest` הגבילו מספר שחקנים ל-2 עד 4.
2. מנעו שמות ריקים או שמות שמכילים רק רווחים.
3. מנעו כפילויות שמות בלי תלות באותיות גדולות/קטנות באנגלית.
4. צרו response models עבור שחקנים, משבצות ומצב משחק.
5. ודאו שכל endpoint מחזיר מבנה JSON עקבי.

בדיקות קבלה

- `test_validation_rejects_bad_player_names` עוברת.
- קלט לא תקין מחזיר HTTP 422.

> **הגשה:** קובץ הסכמות והסבר קצר: למה Pydantic מתאים כאן?

Checkpoint 4 - קובייה, תנועה ומעבר דרך Start

מטרה

לממש זריקת קובייה, תזוזה סביב לוח מעגלי, ובונוס על מעבר במשבצת Start.

קבצים מרכזיים

- `game_engine/services/dice_service.py`
- `game_engine/services/movement_service.py`
- `game_engine/services/game_service.py`

משימות

1. ממשו `DiceService.roll` שמחזיר מספר בין 1 ל-6.
2. ממשו תזוזה לפי גודל הלוח עם חזרה לתחילת הלוח.
3. אם שחקן עבר את Start, הוסיפו לו 200 כסף.
4. אחרי זריקה, עדכנו `available_actions` לפי סוג המשבצת.
5. הוסיפו `POST /api/game/roll` endpoint.

בדיקות קבלה

- בדיקה עם קובייה מדומה מזיזה את השחקן למיקום נכון.
- מעבר ממיקום 14 עם זריקה 3 מסתיים במיקום 1 ומוסיף 200.

הגשה: שירותי הקובייה והתנועה, route של roll, ותוצאות הבדיקות הרלוונטיות.

Checkpoint 5 - ירושה במשבצות וקניית נכסים

מטרה

להשתמש בירושה כדי לייצג סוגי משבצות שונים, ולממש קניית נכס.

משימות

1. צרו מחלקת בסיס `Tile` עם id, name ו-type.
2. צרו `StartTile`, `PropertyTile`, `RewardTile`, `PenaltyTile`, `SurpriseTile`.
3. ל-`PropertyTile` הוסיפו price, rent ו-owner_id.
4. כאשר שחקן נוחת על נכס פנוי, אפשרו פעולה `buy`.
5. ממשו `POST /api/game/buy`: הורדת כסף, סימון בעלים, הוספת הנכס לרשימת השחקן.

בדיקות קבלה

- קניית נכס מורידה את מחיר הנכס מכסף השחקן.
- הנכס מקבל `owner_id` של השחקן.
- לאחר קנייה הפעולה הזמינה היא `end_turn`.

> **הגשה:** מחלקות המשבצות, buy flow, ותשובה לשאלה: איפה בפרויקט רואים ירושה?

Checkpoint 6 - שכירות, בונוס, קנס והפתעה

מטרה

להוסיף אפקטים למשבצות אחרי נחיתה.

קובץ מרכזי

`game_engine/services/tile_service.py`

משימות

1. אם שחקן נוחת על נכס של שחקן אחר, הורידו ממנו rent והוסיפו לבעלים.
2. אם שחקן נוחת על Reward, הוסיפו כסף.
3. אם שחקן נוחת על Penalty, הורידו כסף.
4. אם שחקן נוחת על Surprise, בחרו אפקט אקראי קטן, למשל רווח 100 או הפסד 100.
5. כתבו הודעה מתאימה ל- `event_log` .

בדיקות קבלה

- שכירות עוברת מהשוכר לבעל הנכס.
- Reward מוסיף כסף.
- Penalty מוריד כסף.
- Surprise ניתן לבדיקה באמצעות בחירה אקראית מדומה.

> **הגשה:** שירות המשבצות ותיאור קצר של כל אפקט שמומש.

Checkpoint 7 - תורות, פעולות זמינות ושגיאות

מטרה

להבטיח שהמשחק מתקדם לפי תור, ושלא ניתן לבצע פעולה בזמן לא מתאים.

קבצים מרכזיים

- `game_engine/services/turn_service.py`
- `game_engine/exceptions.py`
- `app/exception_handlers.py`

משימות

1. ממשו `end_turn` שמעביר לשחקן הבא.
2. דלגו על שחקנים שפשטו רגל.
3. השתמשו ב- `available_actions` כדי למנוע פעולה לא חוקית.
4. צרו חריגות דומיין כגון `GameNotFoundError` ו- `InvalidActionError` .
5. חברו exception handlers שמחזירים שגיאת HTTP ברורה.

בדיקות קבלה

- סיום תור אחרי שחקן 1 מעביר לשחקן 2.
- שחקן שפשט רגל לא מקבל תור.
- פעולה לא זמינה לא משנה את מצב המשחק.

> **הגשה:** שירות התורות, חריגות, ותוצאת בדיקות התורות.

Checkpoint 8 - פשיטת רגל, מנצח ואיפוס

מטרה

לסגור את מחזור המשחק המלא: הפסד, ניצחון והתחלה מחדש.

משימות

1. כאשר כסף של שחקן יורד מתחת ל-0, סמנו אותו כ-bankrupt.
2. אם נשאר רק שחקן פעיל אחד, סיימו את המשחק.
3. עדכנו `status` ל-`finished` ושמרו `winner_id`.
4. כאשר המשחק הסתיים, רשימת הפעולות הזמינות ריקה.
5. ממשו `POST /api/game/reset` שמחזיר מצב `not_started`.

בדיקות קבלה

- קנס גדול יכול לגרום לפשיטת רגל.
- כאשר נשאר שחקן אחד פעיל, המשחק מסתיים.
- איפוס מחזיר players ו-board ריקים.

הגשה: מנגנון סיום משחק, reset, והרצה מלאה של `pytest`.

Checkpoint 9 - חיבור לממשק ושיפור חוויית משחק

מטרה

לוודא שהשרת עובד עם ה-React UI וללטש את מידע המשחק שמוחזר למשתמש.

משימות

1. הריצו backend ו-frontend יחד.
2. צרו משחק דרך הממשק.
3. בדקו roll, buy, end turn ו-reset מהממשק.
4. שפרו הודעות `last_event` ו- `event_log` כך שיהיו ברורות לשחקנים.
5. ודאו שאין חישובי חוקי משחק בצד React.

בדיקות קבלה

- ניתן לשחק לפחות 5 תורות דרך הממשק בלי שגיאה.
- הפעולות בכפתורים תואמות ל- `available_actions` .
- ה-log מציג מה קרה בכל פעולה.

הגשה: צילום מסך של משחק פעיל, צילום מסך של event log, והרצה מלאה של בדיקות backend.

מחוון הערכה מוצע

תחום	משקל	מה בודקים
נכונות פונקציונלית	40%	הבדיקות עוברות והמשחק עובד לפי החוקים.
ארכיטקטורה	25%	הפרדה בין routes, schemas, services, models ו-repository.
איכות קוד	20%	שמות ברורים, פונקציות קצרות, ללא כפילויות מיותרות.
עבודה עם API	10%	חוזה JSON יציב ותואם לממשק.
הגשה ותיעוד	5%	תיעוד קצר של מה נעשה ואילו בדיקות הורצו.

שאלות רפלקציה לסיום

1. איזו שכבה בפרויקט אחראית על חוקי המשחק? למה?

2. מה ההבדל בין Pydantic model לבין dataclass של מנוע המשחק?
3. איפה השתמשתם בירושה, ומה היתרון שלה במשבצות הלוח?
4. למה חשוב לבדוק קובייה באמצעות monkeypatch בבדיקות?
5. איזה שינוי הייתם מוסיפים למשחק אם היה עוד זמן?

מסמך זה נועד לשמש כקובץ פתיחה והנחיית checkpoints לפרויקט Property Empire Classroom.